Grafana ⭐ ⭐ ⭐ — Interview-Ready Q&A

1. What is Grafana?
Grafana is an **observability and visualization tool** used to create dashboards and visualize metrics from different data sources.
In our setup, Prometheus collects and stores metrics, and Grafana displays those metrics in dashboards.
Prometheus
↓
Metrics
↓
Grafana
↓
Dashboards
Simple ga gurthupettuko:
Prometheus = **Metrics collect/store**
Grafana = **Metrics visualize**
Eng: Grafana is an open-source observability and visualization platform used to query, visualize, and monitor data from multiple sources.

---

2. Why do we use Grafana?
Raw metrics ni directly choodadam difficult.
Grafana converts those metrics into easy-to-understand dashboards, graphs, tables, and other visualizations.
For example, we can create dashboards showing:

- CPU usage
- Memory usage
- Request rate
- Error rate
- Response time
- Kubernetes Pod status

Simple ga gurthupettuko:
Raw Metrics ❌ → Grafana Dashboard ✅
Eng: Grafana is used to visualize monitoring data in dashboards so engineers can quickly understand system health and performance.

---

3. What is a Data Source in Grafana?
A Data Source is the system from which Grafana retrieves data.
Examples:
Prometheus
CloudWatch
Loki
Elasticsearch
InfluxDB
For our common Prometheus setup:
Grafana → Prometheus → Metrics
Simple ga gurthupettuko:
Data Source = **Grafana data teesukune place.**
Eng: A Grafana Data Source is a configured backend from which Grafana queries data for visualization and monitoring.

---

4. How do you add Prometheus as a Data Source in Grafana?
First, I configure Prometheus as a Data Source in Grafana.
I provide the Prometheus server URL, for example:
http://prometheus:9090
Then I test the connection.
If the connection is successful, Grafana can execute PromQL queries against Prometheus.
Simple ga gurthupettuko:
Grafana → Prometheus URL → Test → Connected
Eng: I configure the Prometheus endpoint as a Grafana Data Source and verify connectivity before creating dashboards.

---

5. What is a Dashboard in Grafana?
A Dashboard is a collection of visualizations that displays monitoring information in one place.
For example, an application dashboard may contain:
Application Dashboard
├— CPU Usage
├— Memory Usage
├— Request Rate
├— Error Rate
└— Response Time
Simple ga gurthupettuko:
Dashboard = **Multiple monitoring panels in one place.**
Eng: A Grafana Dashboard is a collection of panels that provides a consolidated view of system or application metrics.

---

6. What is a Panel in Grafana?
A Panel is an individual visualization inside a dashboard.
For example:
Dashboard
├— CPU Panel
├— Memory Panel
├— Request Panel
└— Error Panel
Each panel can have its own query and visualization type.
Simple ga gurthupettuko:
Panel = **Dashboard lo one graph/view.**
Eng: A Grafana Panel is an individual visualization that displays data returned by a configured query.

---

7. What is the difference between Dashboard and Panel?
A **Dashboard** is the complete monitoring page.
A **Panel** is one visualization inside that dashboard.
Dashboard
├— Panel 1
├— Panel 2
├— Panel 3
└— Panel 4
Simple ga gurthupettuko:
Dashboard = **Collection**
Panel = **Individual visualization**
Eng: A dashboard contains multiple panels, while each panel represents a specific visualization or metric view.

---

8. What is a Query in Grafana?
A Query tells Grafana **what data to retrieve from the Data Source**.
If Prometheus is the Data Source, we commonly use PromQL.
Example:
up
or:
rate(http_requests_total[5m])
Grafana executes the query and displays the result in the selected panel.
Simple ga gurthupettuko:
Query = **Which data kavali ani cheppedi.**
Eng: A Grafana Query retrieves the required data from a configured Data Source for visualization or analysis.

---

9. How does Grafana work with Prometheus?
The flow is:
Application
↓
Prometheus
↓
Metrics stored
↓
Grafana
↓
PromQL Query
↓
Dashboard
Grafana does not normally collect the metrics itself in this setup. It queries Prometheus and visualizes the returned data.
Simple ga gurthupettuko:
Prometheus = **Collect + Store**
Grafana = **Query + Visualize**
Eng: Grafana queries metrics from Prometheus using PromQL and presents the results through dashboards and visualizations.

---

10. What are the common visualization types in Grafana?
Common visualizations include:

- Time series graphs
- Bar charts
- Stat panels
- Gauges
- Tables
- Heatmaps

For example, CPU usage over time is commonly displayed using a time-series panel.
Simple ga gurthupettuko:
Visualization = **Data ni easy ga understand ayye format lo show cheyyadam.**
Eng: Grafana provides multiple visualization types to represent monitoring data in ways that make trends and system conditions easier to understand.

11. What is a Time Series panel?
A Time Series panel displays how a metric changes over time.
For example:
10:00 → CPU 40%
10:05 → CPU 55%
10:10 → CPU 80%
10:15 → CPU 60%
This is very useful for identifying spikes, trends, and performance issues.
Simple ga gurthupettuko:
Time Series = **Metric over time.**
Eng: A Time Series panel visualizes metric values over time and is useful for identifying trends, spikes, and performance changes.

---

12. What are Grafana Variables?
Variables allow users to dynamically change dashboard filters without creating separate dashboards.
For example, instead of creating separate dashboards for:
dev
qa
prod
we can create an Environment variable:
$environment
Then the user can select dev, qa, or prod from the dashboard.
Simple ga gurthupettuko:
Variable = **One dashboard → Different filters/environments.**
Eng: Grafana Variables make dashboards dynamic by allowing users to select values such as environment, service, namespace, or instance.

---

13. Why are Variables useful in Production?
Suppose we have 100 Kubernetes Pods.
Instead of creating 100 dashboards, we can create one dashboard with variables such as:
Environment
Namespace
Service
Pod
The same dashboard can then be used for different workloads.
Simple ga gurthupettuko:
One Dashboard + Variables = **Reusable Dashboard**
Eng: Variables make monitoring dashboards reusable and scalable across multiple environments, services, namespaces, and instances.

---

14. What is an Alert in Grafana?
A Grafana Alert evaluates a condition against query results and can trigger an alert when the condition is met.
For example:CPU > 80%
for a configured evaluation period can trigger an alert.
Grafana can then route the notification through its configured contact points and notification policies.
Simple ga gurthupettuko:
Metric → Condition → Alert → Notification
Eng: Grafana Alerting evaluates defined conditions against query results and can send notifications when alert rules enter a firing state.

---

15. Difference between Grafana Alerting and Prometheus Alerting?
Prometheus can evaluate alert rules against Prometheus metrics and send firing alerts to Alertmanager.
Grafana also has its own alerting system that can evaluate queries from supported data sources and route notifications.
Prometheus Alert:
Prometheus → Alertmanager → Notification
Grafana Alert:
Data Source → Grafana Alert → Notification
Simple ga gurthupettuko:
Prometheus Alerting = **Prometheus-based rules**
Grafana Alerting = **Grafana-based alert rules**
Eng: Prometheus Alerting evaluates Prometheus rules and commonly uses Alertmanager, while Grafana Alerting provides a centralized alerting mechanism that can evaluate queries across supported data sources.

---

16. What is a Grafana Alert Rule?
An Alert Rule defines **when an alert should fire**.
For example:
Query → CPU Usage
Condition → Above 80%
Duration → 5 minutes
If the condition remains true according to the rule's evaluation, Grafana can generate a firing alert.
Simple ga gurthupettuko: Alert Rule = **Eppudu alert fire avvali ani condition.**
Eng: An Alert Rule defines the query, condition, and evaluation behavior used to determine when Grafana should trigger an alert.

17. What is a Contact Point in Grafana?
A Contact Point defines **where an alert notification should be sent**.
Examples can include:
Email
Slack
PagerDuty
Webhook
The exact integrations depend on the Grafana setup and supported notification channels.
Simple ga gurthupettuko:
Contact Point = **Alert ekkadiki pampali.**
Eng: A Grafana Contact Point defines the notification destination used when an alert is triggered.

---

18. What is a Notification Policy?
A Notification Policy determines **how alerts are routed** based on alert labels or other matching criteria.
For example:
Critical → On-call team
Warning → DevOps team
Database → DBA team
This helps avoid sending every alert to every team.
Simple ga gurthupettuko:
Notification Policy = **Which alert → Which team.**
Eng: A Notification Policy controls how Grafana alerts are routed to appropriate notification destinations based on defined matching rules.

---

19. How do you create a Production dashboard?
First, I identify the important application and infrastructure metrics.
For example:
Infrastructure
├── CPU
├── Memory
├── Disk
└── Network

Application
├── Request Rate
├── Error Rate
├── Response Time
└── Availability
Then I configure the appropriate Data Source, create PromQL queries, add panels, and use variables for environment/service filtering.
Simple ga gurthupettuko:
Data Source → Queries → Panels → Variables → Dashboard
Eng: I build Production dashboards around key infrastructure and application health metrics, using reusable queries, panels, and variables.

---

20. What metrics would you monitor for a web application?
I would monitor the **Golden Signals**:
Latency
Traffic
Errors
Saturation
I would also monitor infrastructure metrics such as CPU, memory, disk, and network usage.
For Kubernetes applications, I would additionally monitor Pod restarts, resource usage, replica availability, and workload health.
Simple ga gurthupettuko:
Application → **Traffic + Errors + Latency + Saturation**
Eng: For a web application, I monitor traffic, latency, errors, saturation, availability, and relevant infrastructure or workload health metrics.

---

21. How do you troubleshoot a Grafana dashboard showing No Data?
First, I check whether the Data Source is reachable.
Then I verify:
Data Source→ Query→ Time Range→ Variables→ Metric Availability
If Prometheus is the Data Source, I run the PromQL query directly in Prometheus to determine whether the problem is with the data itself or the Grafana configuration.
Simple ga gurthupettuko: No Data → Data Source → Query → Time Range → Variables → Metrics
Eng: I troubleshoot Grafana No Data issues by checking the Data Source, query, time range, variables, and whether the underlying metrics are actually available.

---

22. What if Grafana cannot connect to Prometheus?
First, I verify the Prometheus URL configured in Grafana.
Then I check:

- Prometheus status
- Network connectivity
- DNS resolution
- Port accessibility
- Authentication/TLS if configured
- Grafana Data Source configuration

I also test the Prometheus endpoint from the Grafana environment when appropriate.
Simple ga gurthupettuko:
Grafana → Prometheus URL → Network → Port → Auth → Test
Eng: If Grafana cannot connect to Prometheus, I verify the endpoint, network connectivity, port, authentication or TLS configuration, and Prometheus availability.

---

23. What if Grafana is showing incorrect data?
First, I verify the query.
Then I check:
Query
↓
Metric
↓
Labels
↓
Variables
↓
Time Range
↓
Aggregation
I compare the same query directly in Prometheus to determine whether the issue is with Prometheus data or Grafana visualization/query configuration.
Simple ga gurthupettuko:
Wrong Data → Query → Labels → Time Range → Aggregation
Eng: I troubleshoot incorrect Grafana data by validating the query, labels, variables, time range, aggregation, and the underlying Data Source.

---

24. How do you monitor Kubernetes using Grafana?
A common setup is:
Kubernetes
↓
Metrics / Exporters
↓
Prometheus
↓
Grafana
↓
Kubernetes Dashboard
We can create dashboards for:
Cluster
Nodes
Namespaces
Pods
Deployments
CPU
Memory
Network
Simple ga gurthupettuko:
Kubernetes → Prometheus → Grafana → Dashboard
Eng: Kubernetes metrics can be collected by Prometheus and visualized in Grafana through dashboards for cluster, node, namespace, and workload monitoring.

---

25. How do you monitor an EC2 server using Grafana?
A common architecture is:

EC2→ Node Exporter→ Prometheus→ Grafana

Node Exporter exposes CPU, memory, disk, filesystem, and network metrics.
Prometheus collects them and Grafana visualizes them.
Simple ga gurthupettuko:
EC2 → Node Exporter → Prometheus → Grafana
Eng: I can monitor an EC2 server by exposing system metrics through Node Exporter, collecting them with Prometheus, and visualizing them in Grafana.

26. What is Dashboard Provisioning?Dashboard provisioning allows Grafana dashboards and related configuration to be managed automatically rather than manually creating everything through the UI.
This is useful when we want consistent dashboards across environments or want to manage Grafana configuration through automation and version control.
Simple ga gurthupettuko:
Manual Dashboard ❌ → **Automated Dashboard Configuration** ✅
Eng: Dashboard provisioning automates the creation and management of Grafana dashboards and helps maintain consistent monitoring configuration.

---

27. How do you manage Grafana dashboards across Dev, QA, and Production?
I prefer to manage dashboards as code or through a controlled provisioning process where practical.
We can use variables for environment-specific filtering and maintain dashboard definitions in version control.
Git
↓
Dashboard Definition
↓
Provisioning / Automation
↓
Grafana
Simple ga gurthupettuko:
Dashboard → Git → Version Control → Automation
Eng: I manage reusable Grafana dashboards through version control and automation so monitoring configuration remains consistent across environments.

---

28. How do you secure Grafana?
I would implement:

- Authentication
- Role-based access control
- HTTPS
- Least-privilege permissions
- Restricted Data Source access
- Secure credentials
- Appropriate dashboard permissions

I would also avoid exposing Grafana directly to the public internet unless the architecture specifically requires it and appropriate security controls are in place.
Simple ga gurthupettuko:
Grafana Security = **Auth + RBAC + HTTPS + Least Privilege**
Eng: I secure Grafana using authentication, authorization, HTTPS, least-privilege access, and controlled network exposure.

---

29. What is the difference between Grafana and Prometheus?
Prometheus is primarily a **metrics collection, storage, querying, and alerting system**.
Grafana is primarily a **visualization and observability platform** that queries data sources and displays dashboards and alerts.

Application → Prometheus → Collect + Store + Query → Grafana → Visualize

Simple ga gurthupettuko:
Prometheus = **Metrics Engine**
Grafana = **Visualization / Dashboard**
Eng: Prometheus primarily collects and stores time-series metrics, while Grafana queries data sources and provides dashboards, visualizations, and alerting capabilities.

30. Explain a real-time Grafana monitoring architecture.
Suppose we have a Production Kubernetes application.
Application and infrastructure metrics are exposed through appropriate metric endpoints or exporters.
Prometheus discovers the targets and scrapes the metrics. Grafana is configured with Prometheus as a Data Source.
Engineers use Grafana dashboards to monitor CPU, memory, traffic, errors, latency, Pods, and other important metrics. Alerts can be configured so critical conditions generate notifications.

Kubernetes / Application→ Metrics / Exporters→ Prometheus→ PromQL→ Grafana→ Dashboards→ Engineer→ Alert Condition→ Grafana Alerting→ Contact Point→ Notification

Simple ga gurthupettuko:
Metrics → Prometheus → Grafana → Dashboard → Alert → Notification
Eng: In a real-time setup, Prometheus collects application and infrastructure metrics, Grafana queries Prometheus and visualizes the data through dashboards, and alerting routes important conditions to the appropriate notification channels.